

Diagrama de Classes da UML

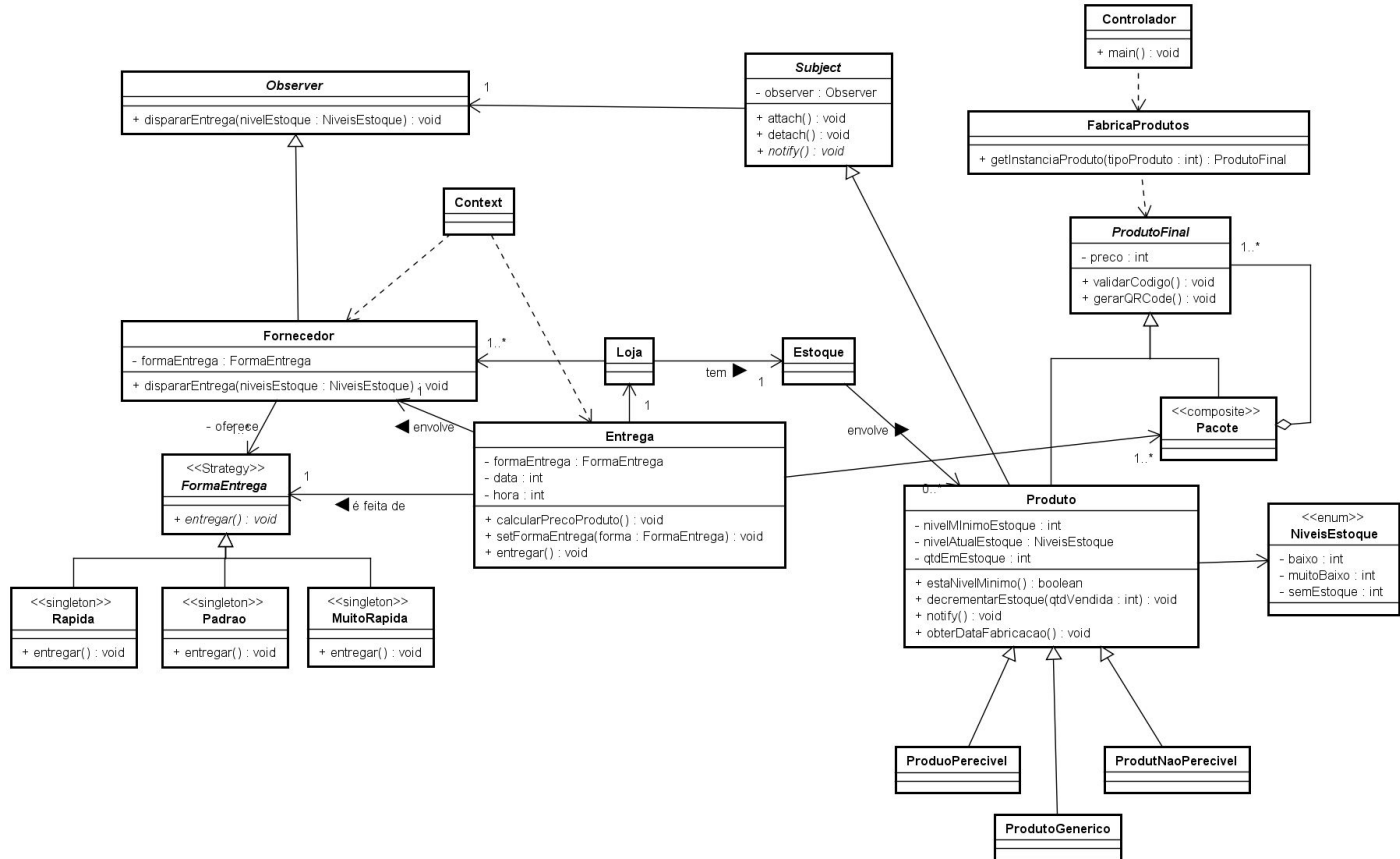
—

P00 2026-1

O que são diagramas de classes ?

Observe:

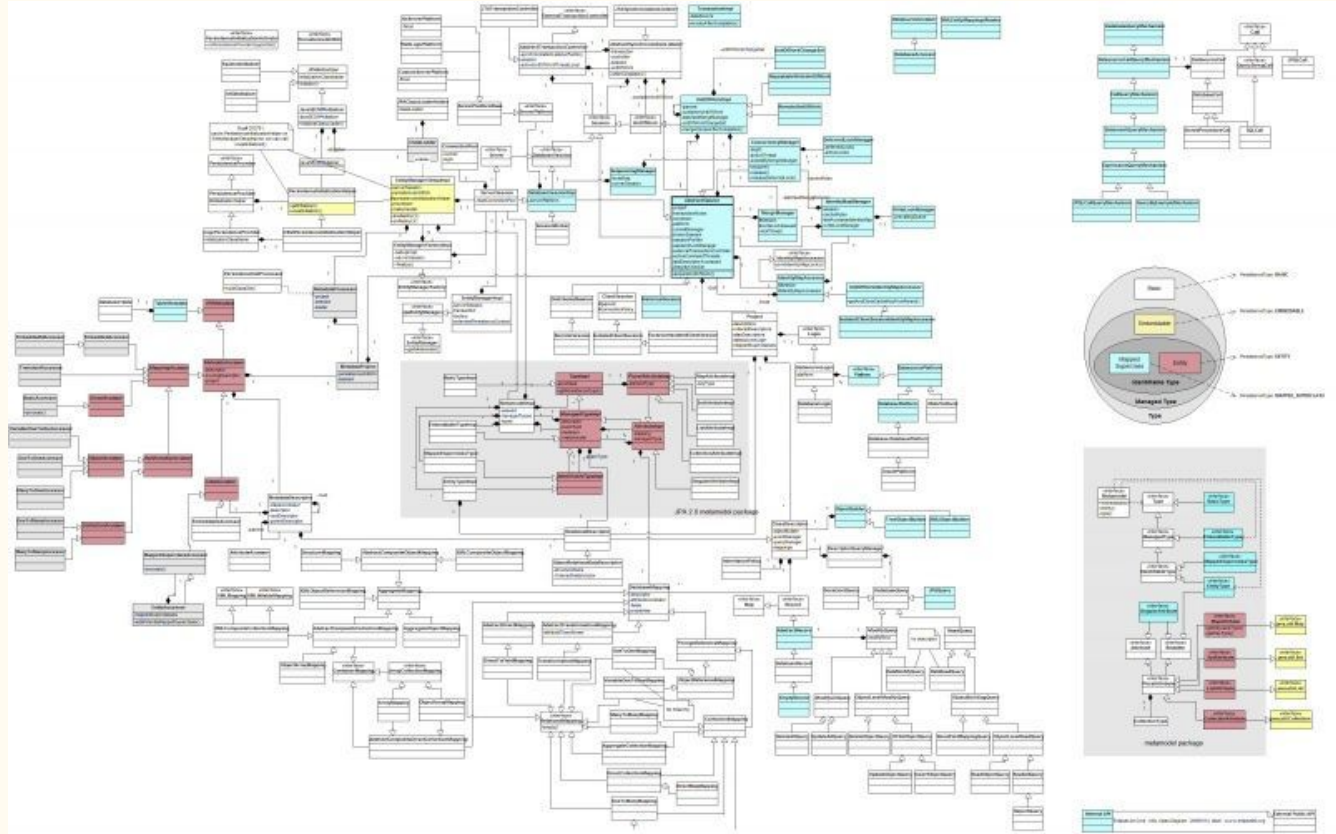
- diferentes tipos de relacionamentos
- tipos de dados
- parâmetros
- multiplicidades
- nomes nos relacionamentos
- ...

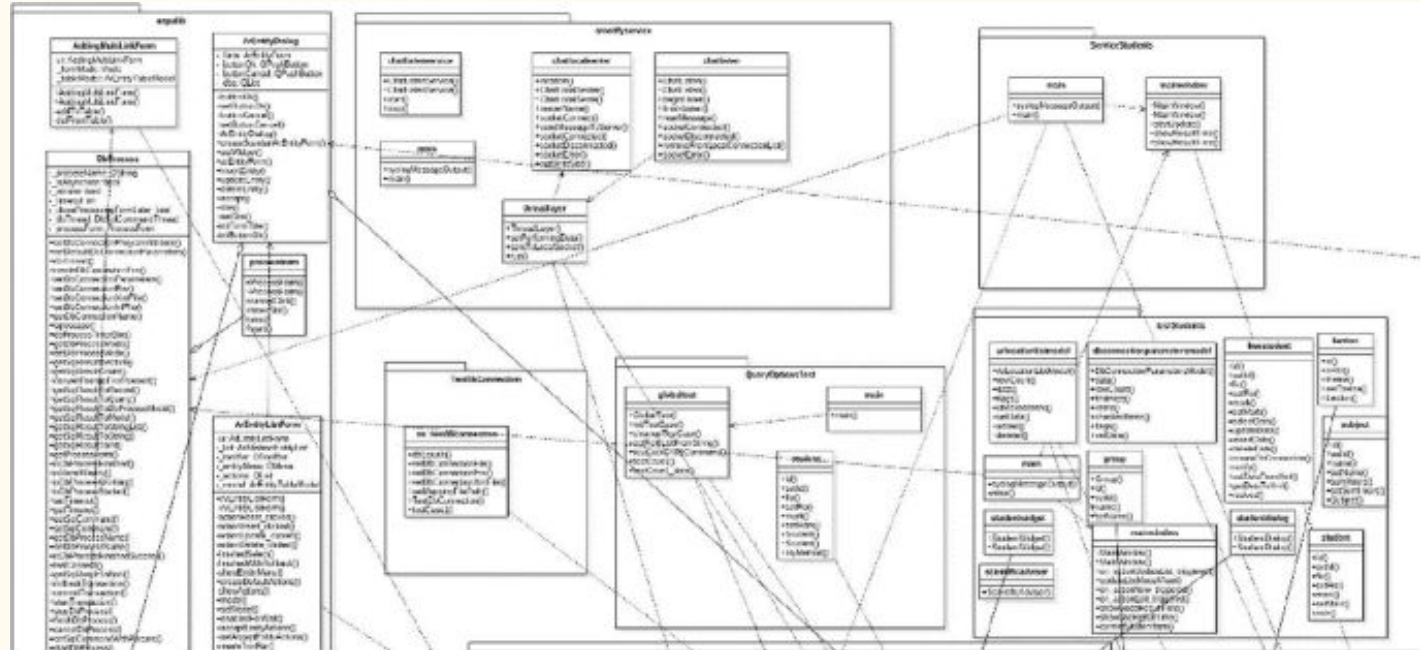


O que são diagramas de classes ?

Observe:

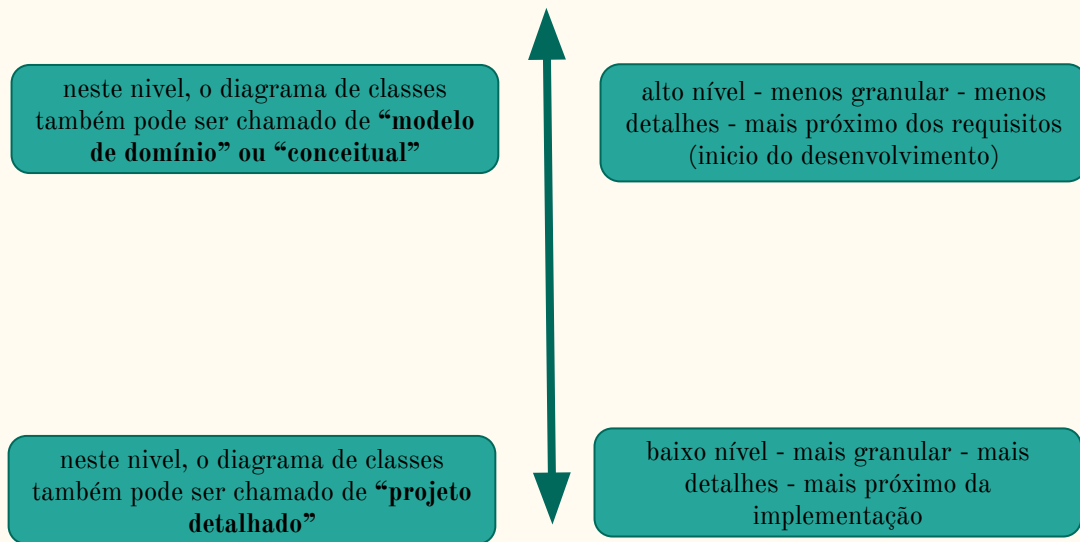
- podem se tornar bem grandes





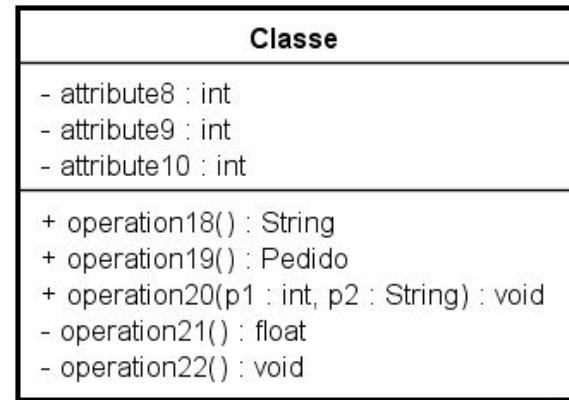
O que é um diagrama de classe ?

- É uma forma de representar o aspecto **estrutural** (classes) de um sistema orientado a objetos
- O aspecto **dinâmico** é capturado pelos diagramas de Sequência e Estado
- Permite diferentes níveis de abstração, mais detalhes ou menos detalhes, dependendo da evolução do entendimento sobre o sistema



Classes

- Uma classe representa um grupo/categoria
- Uma classe é uma **abstração** do mundo real
- Possui atributos (dados) e métodos (comportamento)
- Representada na UML como uma caixa com até três compartimentos
- Se transforma em um elemento de código em alguma linguagem
- Algumas possuem uma tabela de banco que a representa
 - As vezes essas classes são chamadas de “entidades”
- Também podem ser vistas como “Tipos Complexos” de dados - na verdade são tipos de dados
- **Objetos** são criados delas - Objetos são variáveis
- Podem ser abstratas
 - representa uma abstração
 - normalmente usadas em hierarquias de herança
 - não podem ser instanciadas



powered by Astah 

Relacionamentos entre classes

—

Relacionamentos

- Em Orientação a Objetos, é dito que os objetos colaboram entre si para a realização de uma determinada funcionalidade
- Essas “colaborações” nada mais são do que os relacionamentos que existem entre as classes
- Um relacionamento, em termos de implementação, é uma **referência** que uma classe tem para uma outra e que permite, por meio dele, acesso a membros dessa outra classe (por ex., chamar métodos)
- Cuidado com a representação gráfica da UML para cada tipo de relacionamento
- Cada tipo de relacionamento possui uma **semântica** que deve ser bem observada

Relacionamentos

→ Associações

- Associação é o tipo mais comum de relacionamento entre classes
- É um relacionamento **estrutural forte** entre duas classes, pois elas são ligadas por seus dados (atributos)
 - Isso significa que uma classe possui um atributo do tipo de outra (na implementação)
- Quando o relacionamento entre as classes A e B é de associação ?
 - Quando é difícil de imaginar um objeto de A sem que B seja parte dele, ou vice-versa
 - Quando não é uma herança
 - Quando não é agregação/composição
- Associações possuem dois subtipos (vistos mais adiante):
 - agregações
 - composições

Relacionamentos

→ Associações

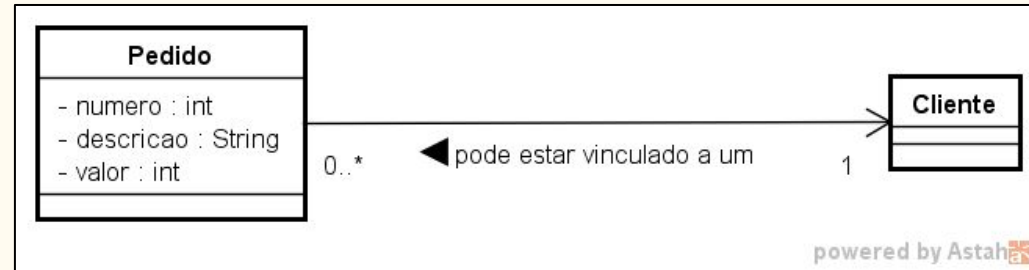
- Difícil imaginar a existência de objeto Pedido sem que exista um cliente atrelado àquele pedido, isso é uma “associação”

Atenção: Isso significa que na classe Pedido há um atributo do tipo Cliente

```
public class Pedido {
```

```
    private Cliente cli;  
    private int numero;  
    private String descricao;  
    private float valor;  
    public void m(){  
        ....  
        cli.chamaAlgumMetodo();  
        ....  
    }  
}
```

Alguns usam o termo “composição” para isso



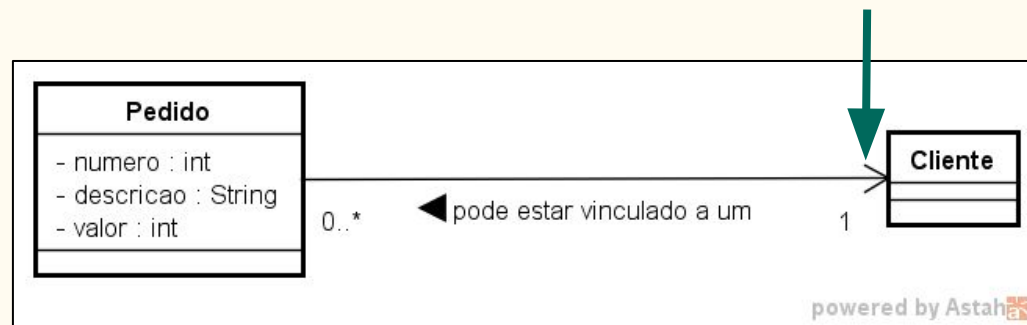
Relacionamentos

→ Associações

- A seta indica “navegabilidade”, isto é, é possível navegar de um objeto Pedido para um objeto Cliente, mas não o contrário.
- Isso também é possível de ser observado pelo atributo cli em Pedido
- Se não é navegável a partir de Cliente, então, não existe um atributo do tipo Pedido nessa classe

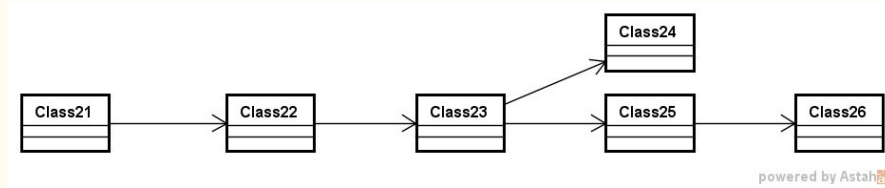
Atenção: Isso significa que na classe Pedido há um atributo do tipo Cliente

```
public class Pedido {  
  
    private Cliente cli;   
  
    public void m(){  
        ....  
        cli.chamaAlgumMetodo();  
        ....  
    }  
    public Cliente getCliente) {  
        return cli;  
    }  
}
```



Relacionamentos

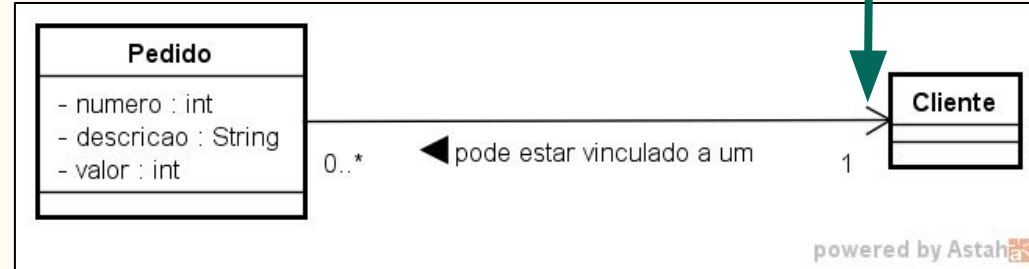
→ Associações



- Navegabilidade é algo importante, mostra como determinadas classes podem ser alcançadas a partir de outras

Atenção: Isso significa que na classe Pedido há um atributo do tipo Cliente

```
public class Pedido {  
    private Cliente cli;  
    public void m(){  
        ....  
        cli.chamaAlgumMetodo();  
        ....  
    }  
    public Cliente getCliente() {  
        return cli;  
    }  
}
```



Relacionamentos

→ Associações

- Se pudesse existir um objeto pedido atrelado a mais do que um cliente, haveria uma lista de clientes na classe Pedido

Atenção: Isso significa que na classe Pedido há um atributo do tipo Cliente

```
public class Pedido {
```

```
    private ArrayList<Cliente> listaCli;
```

```
    public void m(){
```

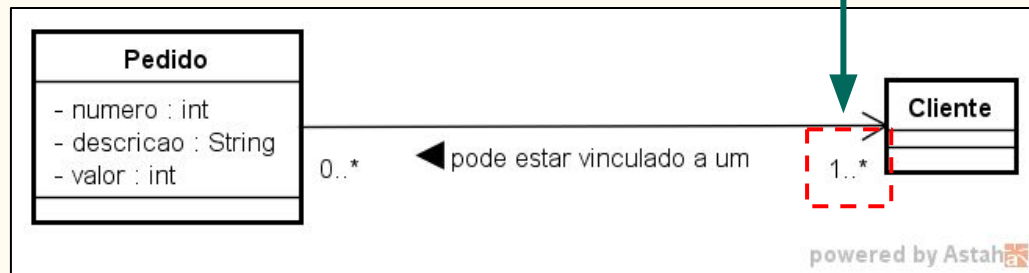
```
        ....
```

```
        ...
```

```
        ....
```

```
    }
```

```
}
```



Relacionamentos

→ Associações - Multiplicidades

- Representam a informação dos limites inferior e superior da quantidade de objetos aos quais outro objeto pode se associar.
- Cada associação em um diagrama de classes possui duas multiplicidades, uma em cada extremo da linha de associação.

Nome

Apenas Um

Zero ou Muitos

Um ou Muitos

Zero ou Um

Intervalo Específico

Simbologia na UML

1..1 (ou 1)

0..* (ou *)

1..*

0..1

$L_i \dots L_s$

Relacionamentos

→ Associações - Multiplicidades



•Exemplo

- Pode haver um cliente que esteja associado a vários pedidos.
- Pode haver um cliente que não esteja associado a pedido algum.
- Um pedido está associado a um, e somente um cliente (sempre). Isto é, não há objeto pedido sem um objeto Cliente.

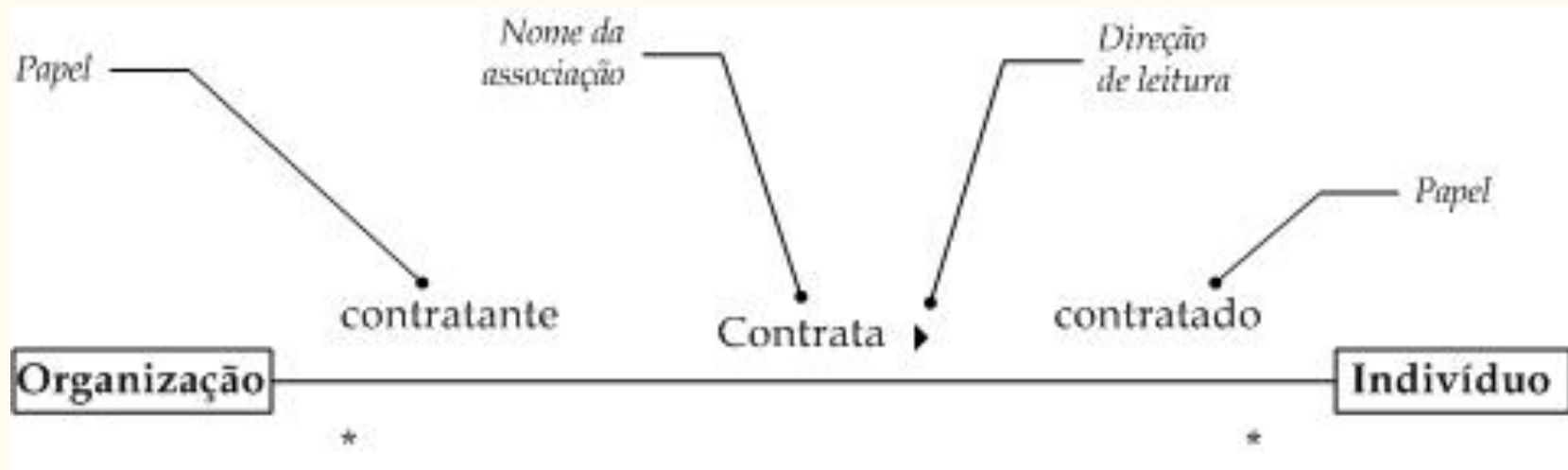


•Exemplo

- Uma corrida está associada a, no mínimo, dois velocistas
- Uma corrida está associada a, no máximo, seis velocistas.
- Um velocista *pode* estar associado a nenhuma ou a várias corridas.

Relacionamentos

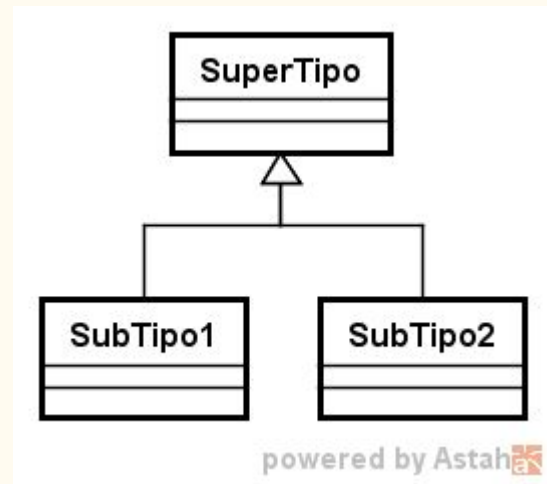
→ Associações - Adornos



Relacionamentos

→ Herança/Generalização

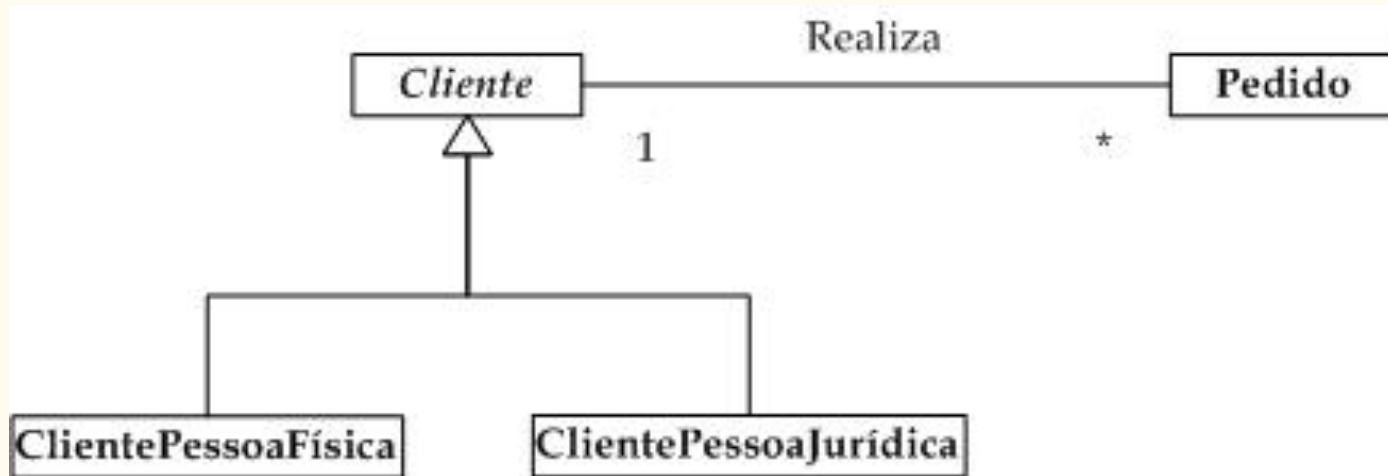
- Usado para representar hierarquia e subtipos
- SuperTipo geralmente é uma classe abstrata, uma abstração para as classes que estão embaixo
- Normalmente, identifica-se um relacionamento de herança fazendo-se a pergunta “É um” ?
 - Se a resposta for “sim”, possivelmente é um relacionamento de herança
 - Cliente físico é um Cliente ?
 - Carro é um Veículo ?
 - Vendedor comissionado é um Vendedor ?
 - Leão é um animal ?
- Herança significa que subclasses herdam as características de sua superclasse



Relacionamentos

→ Herança/Generalização

- Não somente atributos e operações, mas também associações são herdadas pelas subclasses.
- No exemplo abaixo, por herança, cada subclasse está associada a Pedido.

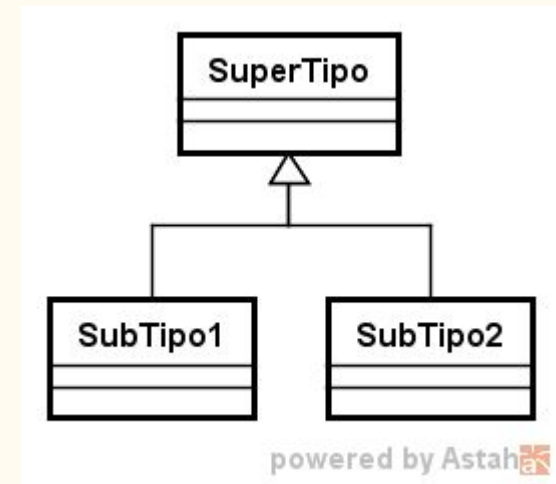


Relacionamentos

→ Herança/Generalização

Terminologia

- superclasse X subclasse
- supertipo X subtipo
- classe base X classe herdeira
- classe de generalização X classe de especialização
- ancestral e descendente (herança em vários níveis)



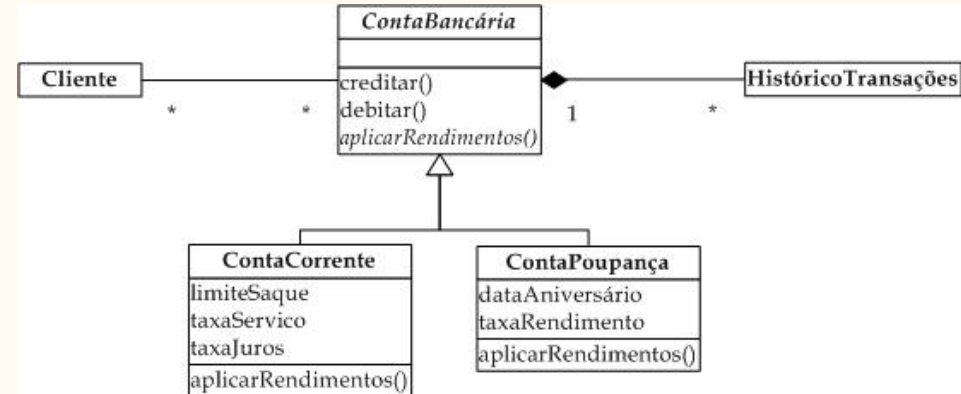
Todos significam a mesma coisa

Relacionamentos

→ Herança/Generalização - Classes Abstratas

- Classes abstratas são utilizadas para organizar e simplificar uma hierarquia de generalização.
- Propriedades comuns a diversas classes podem ser organizadas e definidas em uma classe abstrata, a partir da qual as primeiras herdam.
- Subclasses de uma classe abstrata também podem ser abstratas, mas a hierarquia deve terminar em uma ou mais classes concretas.
- Na UML elementos em *Itálico* são abstratos

Conselho: Quase sempre utilizem classes abstratas como classes-base de uma hierarquia. A exceção é usar classe concretas.



FIM